МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ "ЛЬВІВСЬКА ПОЛІТЕХНІКА"
Кафедра «Систем автоматизованого проектування»



ЗВІТ

до лабораторної роботи №9

з дисципліни "Веб-розробка та дизайн"

на тему:

«Адаптивні форми зворотного зв'язку та клієнтська валідація»

Освітній веб-проєкт: SimpleArduino

Тематичний варіант: «Онлайн-освіта / Інтерактивна платформа курсів з робототехніки та Arduino»

Публічний URL сайту (Vercel): https://simplearduino-lab8.vercel.app

Публічний URL сайту (GitHub Pages): https://nazarbuliga.github.io/simplearduino-lab8/

Репозиторій коду (GitHub): https://github.com/nazarbuliga/simplearduino-lab8

Виконав:
студент групи ПП-25
Булига Назарій

Перевірила:
доцент кафедри САПР
Паращин Ж.Д.

Львів – 2026

Мета роботи:

Опанувати принципи семантичної та адаптивної верстки сучасних елементів вебформ за допомогою Tailwind CSS. Вивчити механізми клієнтської валідації введення даних за допомогою регулярних виразів (RegEx) та мови JavaScript. Навчитися проєктувати логіку обробки помилок та інтерактивних UX/UI станів (включаючи кастомні модальні вікна) в синергії з ШІ-інструментами генерації коду й компонентів, а також інтегрувати форму в реальну вебплатформу з контролем версій Git та деплоєм на Vercel.

Об'єкт розробки:

Освітній інтерактивний веб-хаб «SimpleArduino» — спеціалізована мультисторінкова вебплатформа для початківців та студентів інженерних спеціальностей, призначена для покрокового освоєння мікроелектроніки, схемотехніки та програмування Arduino (C/C++). В межах Лабораторної роботи №9 розроблено та інтегровано високоякісну адаптивну форму зворотного зв'язку, що надає можливість користувачам звертатися до менторів курсу, обирати навчальний напрямок (Blink, Сенсори, FreeRTOS, IoT), проходити сувору клієнтську перевірку контактних даних за українським стандартом зв'язку (+380...) та отримувати миттєвий відгук через кастомне модальне вікно без перезавантаження сторінки.

Стек інструментів для виконання роботи:

Категорія	Інструмент	Призначення та застосування в ЛР
Редактор коду	**VS Code**	Створення структури проєкту, написання HTML5/JS/CSS коду, плагіни Tailwind IntelliSense
CSS-фреймворк	**Tailwind CSS v3+**	Побудова мобільної (1 колонка) та десктопної (2 колонки) сіток, UI-станів :focus, ring
Мова логіки	**JavaScript (ES6+)**	Клієнтська валідація, події input/blur/submit, e.preventDefault(), керування модалкою
ШІ-асистенти	**Antigravity / Gemini / Claude**	Role-based prompting, заземлення контексту, генерація RegEx, перевірка Edge Cases
Контроль версій	**Git & GitHub**	Фіксація коммітів за Conventional Commits, ведення гілок, публічний хмарний репозиторій
Хмарний деплой	**Vercel & Netlify**	Автоматизований CI/CD деплой статичного сайту при пуші в репозиторій GitHub

ХІД ВИКОНАННЯ РОБОТИ

Етап 1. Аналіз завдання та проєктування структури форми

Перед безпосереднім написанням коду було проведено ретельний інженерний аналіз бізнес-вимог до форми зворотного зв'язку освітньої платформи SimpleArduino. Основна функція форми — надати можливість студентам та початківцям у сфері мікроелектроніки швидко зв'язатися з технічними менторами для консультації щодо збирання схем на платах Arduino Uno / Mega / Nano, налагодження програмного коду на C/C++ або отримання довідки про наявність радіодеталей у львівському магазині-партнері «Юстас».

Згідно з технічним регламентом лабораторної роботи, форма містить обов'язкові інтерактивні вузли: поле імені користувача, поле адреси електронної пошти, поле мобільного телефону в національному форматі України, селектор теми навчального модуля, текстову область для розгорнутого опису проблеми та семантичний чекбокс обов'язкового підтвердження згоди з Політикою конфіденційності.

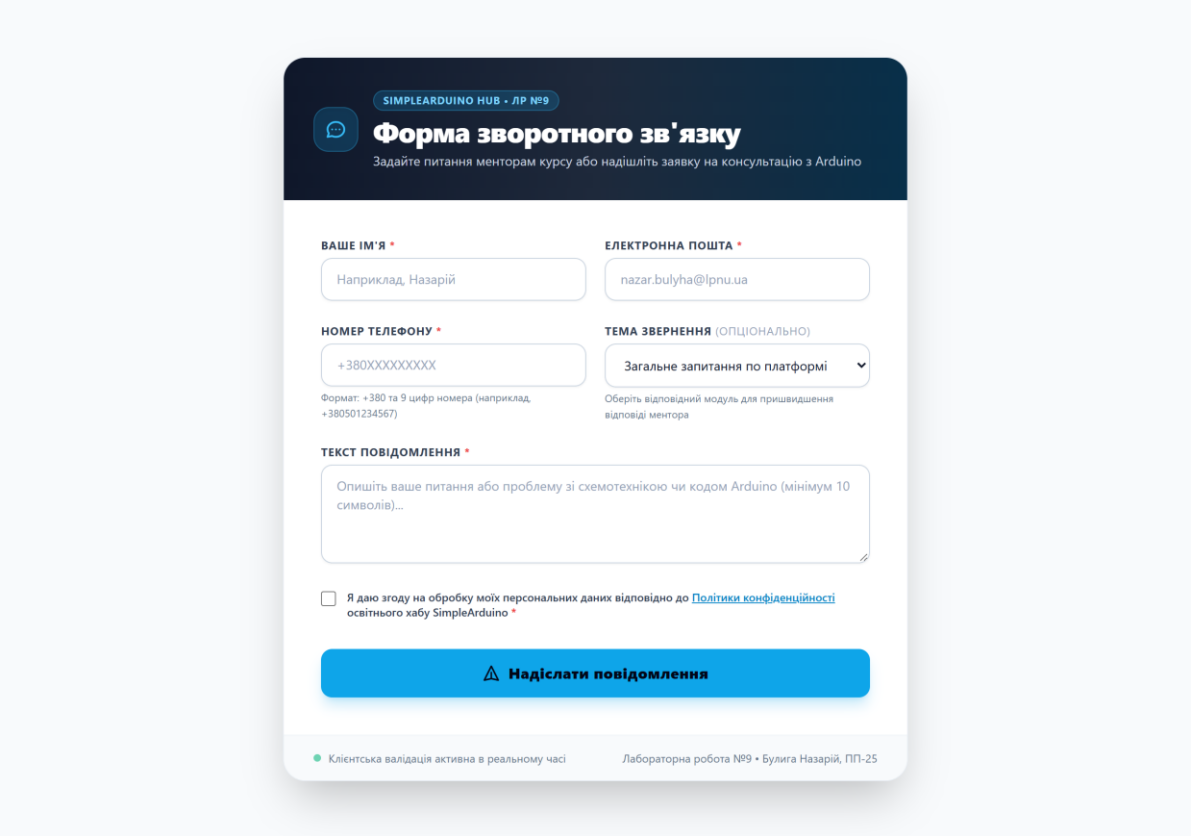


Рис. 1. Архітектура та спроєктований інтерфейс адаптивної форми зворотного зв'язку SimpleArduino у початковому стані

Етап 2. Підготовка структури проєкту

Для забезпечення модульності та зручності підтримки було реалізовано дворівневу архітектуру: окремий автономний модуль форми у каталозі feedback-form/ для швидкого ізольованого тестування та інтегрований варіант форми у головному лендингу SimpleArduino (секція контактів поряд із картою).

```
lab-08-deployment/
│
├── index.html                    # Головний адаптивний лендинг SimpleArduino (секція
контактів)
├── js/
│   ├── main.js                   # Інтерактивна логіка лендингу (фільтри, телеметрія
IoT)
│   └── validation.js             # Модуль клієнтської валідації форми та обробки
модалки
├── css/
│   ├── style.css                 # Загальні стилі проєкту
│   └── custom-styles.css         # Стилізація інтерактивної карти Google Maps
│
└── feedback-form/                # Автономний стенд компонента форми
    ├── index.html                # Семантичний каркас форми зворотного зв'язку
    └── assets/
        ├── css/
        │   └── style.css         # Кастомні UI-стани, анімації та WCAG фокус
        └── js/
            └── validation.js     # Повна клієнтська логіка, RegEx та Edge Cases
```

Етап 3. Проєктування семантичної HTML-структури та доступності

Семантична структура форми базується на сучасних стандартах консорціуму W3C. Тег <form> містить атрибут novalidate, який свідомо відключає неконтрольовані стандартні браузерні спливаючі підказки, передаючи повний контроль над UX клієнтським скриптам JavaScript. Для кожного поля введення створено суворий зв'язок <label for="..."> ↔ <input id="...">, що гарантує збільшення зони кліку та коректну роботу скрінрідерів. Поля обов'язкового заповнення помічені семантичним атрибутом aria-required="true", а контейнери помилок пов'язані з відповідними інпутами за допомогою атрибуту aria-describedby.

```
<!-- Фрагмент семантичної верстки зв'язку Label - Input та блоку помилки -->
<div>
  <label for="username" class="block text-xs font-bold uppercase tracking-wider text-
slate-700 mb-1.5">
    Ваше ім'я <span class="text-red-500 font-bold" aria-hidden="true">*</span>
  </label>
  <div class="relative rounded-xl shadow-sm">
    <input
      type="text"
      id="username"
```

```
      name="username"
      required
      minlength="2"
      autocomplete="name"
      placeholder="Наприклад, Назарій"
      aria-required="true"
      aria-describedby="nameError"
      aria-invalid="false"
      class="form-input-transition block w-full px-4 py-3 rounded-xl border border-
slate-300 text-sm ..."
    >
    <div id="usernameStatusIcon" class="hidden absolute inset-y-0 right-0 pr-3.5 flex
items-center ..." aria-hidden="true">
      <svg class="w-5 h-5 text-emerald-500" fill="currentColor" viewBox="0 0 20
20">...</svg>
    </div>
  </div>
  <div id="nameError" class="error-message hidden" role="alert" aria-live="polite">
    <svg class="w-3.5 h-3.5 flex-shrink-0" fill="currentColor" viewBox="0 0 20
20">...</svg>
    <span id="nameErrorText">Ім'я повинно містити щонайменше 2 символи.</span>
  </div>
</div>
```

Етап 4. Адаптивна верстка за допомогою Tailwind CSS

Верстка форми виконана за методологією Mobile-First. На мобільних екранах (< 640px) усі елементи шикуються у вертикальну 1-колонкову послідовність (grid-cols-1) із висотою полів не менше 48px, що відповідає стандарту WCAG щодо ергономіки сенсорних дисплеїв. На ширших екранах (десктопи та планшети sm:) сітка трансформується у 2 колонки (sm:grid-cols-2 gap-5), логічно групуючи персональні дані («Ім'я» та «Email») і контактно-тематичні поля («Телефон» та «Тема звернення»). Текстова область повідомлення та кнопка відправлення займають повну ширину картки.

Етап 5. Проєктування системи візуальних UI-станів форми

Форма функціонує як скінченний автомат інтерактивних станів, що дає користувачеві миттєвий зворотний зв'язок на кожному кроці заповнення:

Стан UI	Класи Tailwind / CSS	Візуальний прояв та реакція інтерфейсу
1. Normal	border-slate-300 bg-white	Спокійний стан, нейтральна світло-сіра рамка, контрастний плейсхолдер
2. Focus	ring-2 ring-sky-500 border-sky-500	Поле стає активним, з'являється м'яке акцентне кільце підсвічування

Стан UI	Класи Tailwind / CSS	Візуальний прояв та реакція інтерфейсу
3. Error	border-red-500 bg-red-50/40 text-red-600	Червона рамка, поява тексту помилки та SVG-іконки, aria-invalid="true"
4. Success	border-emerald-500 ring-emerald-400	Смарагдова межа, приховування помилки, поява галочки успіху праворуч

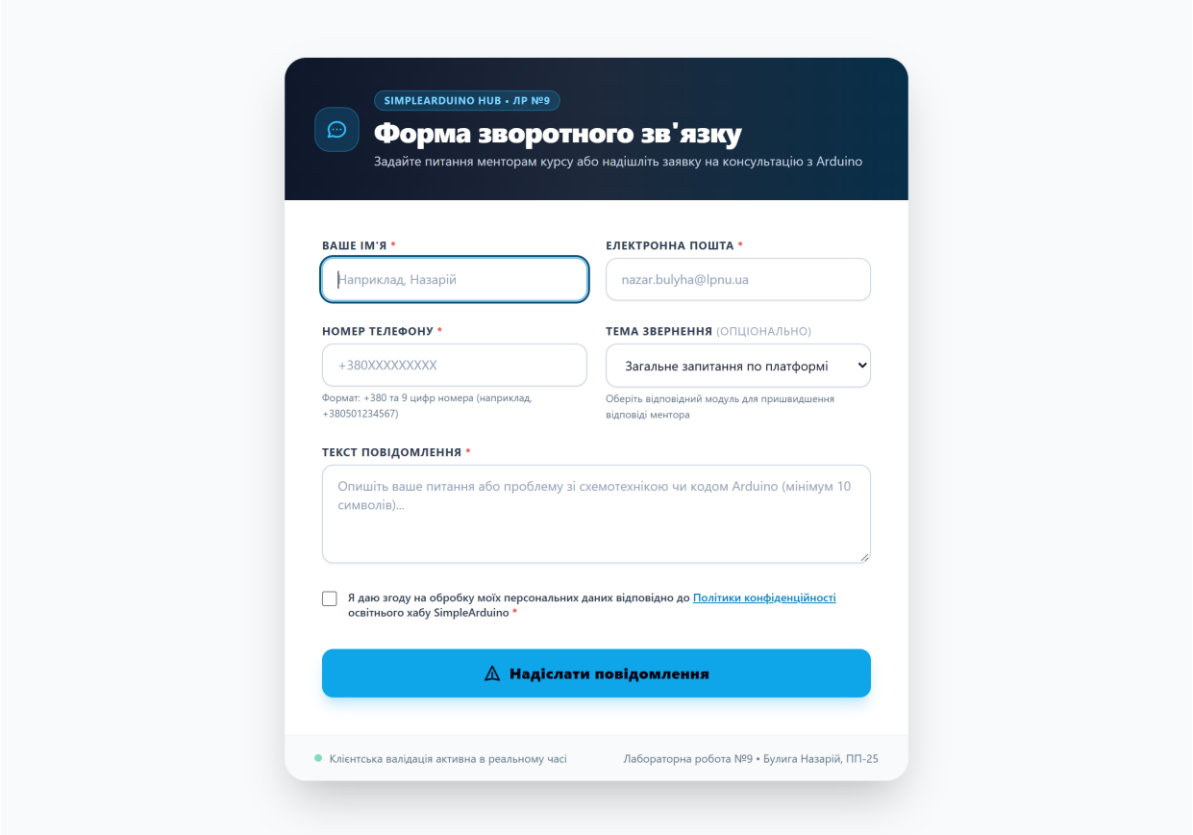


Рис. 2. Стан фокусування (Focus State) на полі введення імені з акцентним контуром ring-sky-500

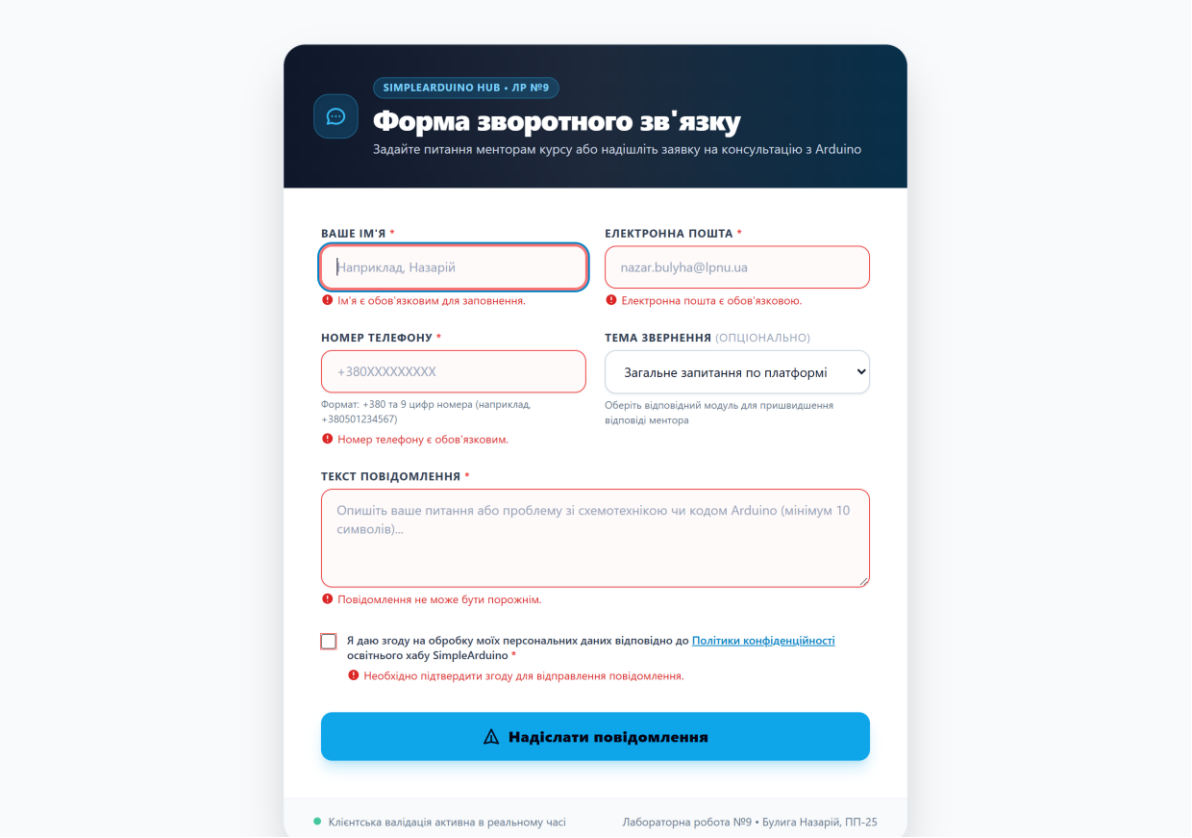


Рис. 3. Стан помилок валідації (Error State) при спробі відправлення порожньої форми

Етап 6. Визначення та систематизація бізнес-правил валідації

Перед написанням логіки JavaScript було створено сувору матрицю бізнес-правил валідації для запобігання надсиланню некоректних даних:

Поле	Тип	Технічне правило	Повідомлення про помилку	Зразок вхідних даних
Ім'я	text	Не пусте, ≥2 символів, літери/апостроф	Ім'я має містити щонайменше 2 символи	Назарій, O'Connor
Емейл	email	Стандарт RFC 5322 (Regex: user@domain.tld)	Введіть коректну адресу (user@mail.com)	nazar.bulyha@lpnu.ua
Телефон	tel	Український стандарт: +380 та 9 цифр	Формат строго український: +380XXXXXXXXX	+380501234567
Повідомлення	textarea	Не пусте, довжина тексту ≥ 10 символів	Повідомлення занадто коротке (мін. 10)	Питання по проєкту 5...
Згода	checkbox	Обов'язковий прапорець (checked == true)	Необхідно надати згоду для відправки	Прапорець встановлено

Етап 7. Валідація за допомогою регулярних виразів (RegEx)

Регулярні вирази є фундаментом клієнтської лінії перевірки структурованих рядків. У роботі застосовано три вирази, кожен із яких було детально проаналізовано:

1. Регулярний вираз для адреси електронної пошти: const REGEX_EMAIL = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;
• ^ та $ — метасимволи початку та кінця рядка. Вони гарантують, що рядок буде перевірено повністю, а не знайдено валідний фрагмент у сміттєвому тексті;
• [a-zA-Z0-9._%+-]+ — дозволяє ім'я поштової скриньки з літер, цифр, знаків відсотка, дефісів і крапок;
• @ — обов'язковий розділювальний знак комерційного «ет»;
• [a-zA-Z0-9.-]+ — назва доменного вузла;
• \. — обов'язкове екранування крапки зворотним слешем. Оскільки крапка в синтаксисі RegEx є метасимволом, що означає «будь-який довільний символ», слеш нейтралізує цю спецфункцію і шукає звичайну граматичну крапку;
• [a-zA-Z]{2,} — квантифікатор діапазону, що вимагає щонайменше 2 літери доменної зони (.ua, .com, .tech).

2. Регулярний вираз для українського номера телефону: const REGEX_PHONE = /^\+380\d{9}$/;
• ^\+380 — рядок обов'язково повинен починатися з екранованого знака «+» та телефонного коду України 380;
• \d{9} — квантифікатор точної кількості, який вимагає рівно 9 десяткових цифр (національний код оператора зв'язку та номер абонента);
• $ — кінець рядка, що унеможливлює введення додаткових прихованих символів чи літер.

Етап 8. Модульна архітектура валідації JavaScript

Логіку перевірки реалізовано у файлі assets/js/validation.js. Замість монолітного коду застосовано архітектуру мікро-функцій, де кожне поле має власний ізольований валідатор:

```
// Допоміжні UI-функції відображення станів
function showErr(inputElement, errorElement, iconElement, message) {
  inputElement.classList.remove('border-slate-300', 'border-emerald-500');
  inputElement.classList.add('border-red-500', 'bg-red-50/40');
  inputElement.setAttribute('aria-invalid', 'true');
  if (errorElement) {
    const textSpan = errorElement.querySelector('span') || errorElement;
    textSpan.textContent = message;
```

```
    errorElement.classList.remove('hidden');
  }
  if (iconElement) iconElement.classList.add('hidden');
}

function showSuccess(inputElement, errorElement, iconElement) {
  inputElement.classList.remove('border-slate-300', 'border-red-500', 'bg-red-50/40');
  inputElement.classList.add('border-emerald-500');
  inputElement.setAttribute('aria-invalid', 'false');
  if (errorElement) errorElement.classList.add('hidden');
  if (iconElement) iconElement.classList.remove('hidden');
}

// Функція валідації номера телефону за стандартом +380XXXXXXXXX
function validatePhone() {
  const input = document.getElementById('phone');
  const error = document.getElementById('phoneError');
  const icon = document.getElementById('phoneStatusIcon');
  if (!input) return true;
  const value = input.value.trim();

  if (value.length === 0) {
    showErr(input, error, icon, "Номер телефону є обов'язковим.");
    return false;
  }
  if (!value.startsWith('+380')) {
    showErr(input, error, icon, "Номер повинен починатися з міжнародного префікса
+380.");
    return false;
  }
  if (value.length < 13) {
    showErr(input, error, icon, `Недостатньо цифр: введено ${value.length - 4} з 9
цифр номера.`);
    return false;
  }
  if (!REGEX_PHONE.test(value)) {
    showErr(input, error, icon, "Формат має бути строго українським: +380XXXXXXXXX
(тільки цифри).");
    return false;
  }
  showSuccess(input, error, icon);
  return true;
}
```

Етап 9. Динамічна валідація без передчасного роздратування користувача

Поширеною помилкою неякісних вебформ є миттєва поява червоних рамок при першому переході користувача у поле. Для вирішення цієї проблеми використано патерн відстеження стану через об'єкт touchedFields. Помилка не відображається, доки користувач не завершив введення і не покинув поле (подія blur). Як тільки поле отримало статус touched, підключається слухач події input, який миттєво знімає червону помилку в той момент, коли значення стає коректним.

Етап 10. Обробка номера телефону та Граничні випадки (Edge Cases)

Особливу увагу приділено обробці нестандартних сценаріїв взаємодії з полем телефону за допомогою функції setupPhoneUX:

• Автодоповнення міжнародного префікса: якщо користувач за звичкою вводить першу цифру 0, система автоматично підставляє +380;
• Санітизація вставки з буфера (Paste): при копіюванні номерів із месенджерів або контактних книжок у форматах на зразок «(050) 123-45-67» або «+38 050 123 45 67», регулярний вираз .replace(/[\s\(\)\-\.]/g, '') видаляє весь сторонній шум і нормалізує рядок до +380501234567;
• Обмеження переповнення: довжина номера програмно обрізається до 13 символів (префікс і 9 цифр).

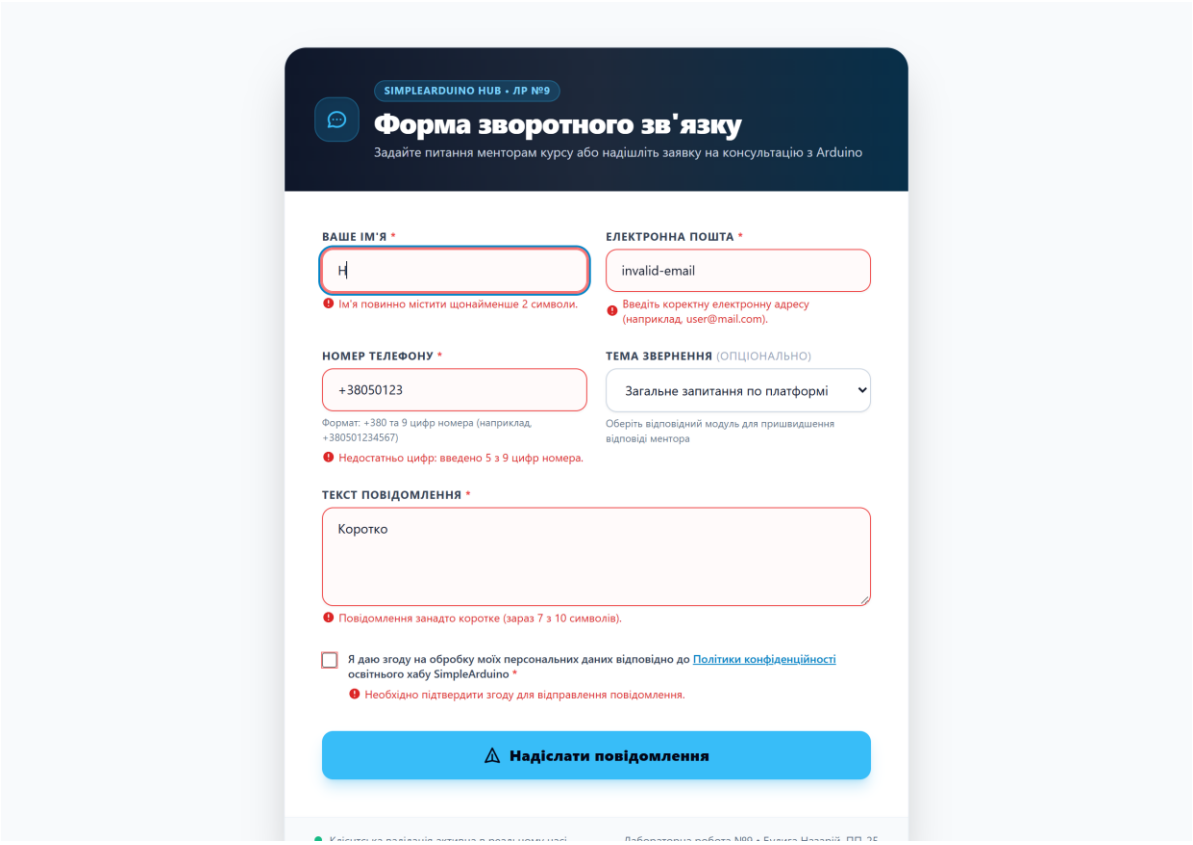


Рис. 4. Демонстрація діагностики граничних випадків (Edge Cases): неповний телефон з динамічним підрахунком цифр, ім'я з 1 літери та пошта без домену

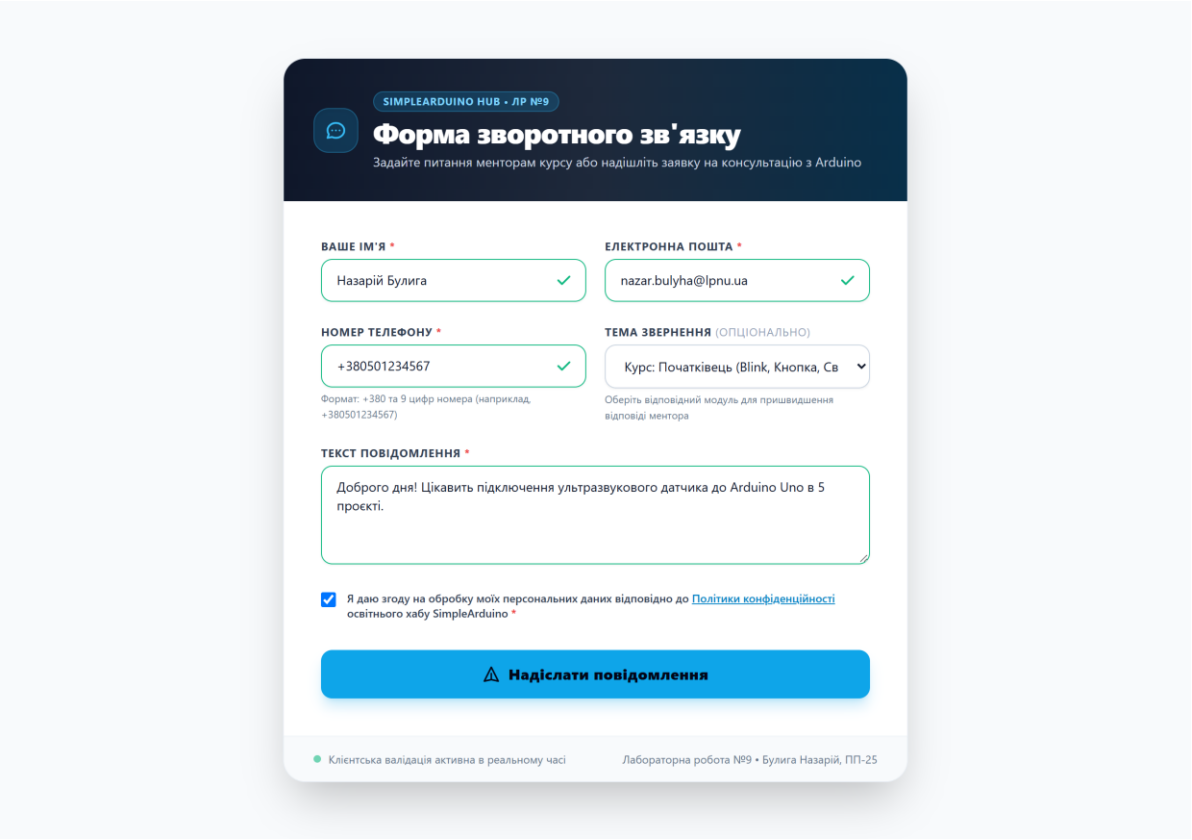


Рис. 5. Стан успішної валідації (Success State) усіх полів із зеленими рамками та галочками підтвердження

Етап 11. Інтеграція FormSubmit AJAX API для доставки на пошту та доступне модальне вікно

Після успішного проходження клієнтської валідації подія відправлення форми перехоплюється через e.preventDefault(). Замість архаїчного window.alert() чи звичайної локальної заглушки, у проєкті впроваджено повноцінну серверну інтеграцію відправлення повідомлень за допомогою FormSubmit AJAX API на персональну академічну скриньку розробника nazarii.bulyha.pp.2025@lpnu.ua:

1. Асинхронний POST-запит: форма відправляє JSON-корисне навантаження на кінцеву точку https://formsubmit.co/ajax/nazarii.bulyha.pp.2025@lpnu.ua з параметром форматування листа _template: table;
2. Динамічний стан завантаження (Loading State): кнопка відправки негайно блокується (disabled = true), змінює текст на «Надсилання...» та активує плавний SVG-спінер обертання, що унеможливлює повторні паразитичні кліки (дебаунсинг);
3. Інтерактивне модальне вікно підтвердження: при отриманні успішної відповіді response.ok спрацьовує відкриття кастомного модального вікна з затемненням фону (backdrop-blur-md), поля форми скидаються (form.reset()),

а користувач бачить персоналізоване повідомлення про успішну доставку; 4. Надійний резервний сценарій (Fail-Safe Fallback): у випадку відсутності інтернет-з'єднання або серверного тайм-ауту система автоматично відкриває стандартний поштовий клієнт через протокол mailto:nazarii.bulyha.pp.2025@lpnu.ua із попередньо заповненою темою та тілом звернення.

```
// Відправлення форми через FormSubmit AJAX на nazarii.bulyha.pp.2025@lpnu.ua
const FORMSUBMIT_URL = 'https://formsubmit.co/ajax/nazarii.bulyha.pp.2025@lpnu.ua';

const formData = {
  name: nameVal,
  email: emailVal,
  phone: phoneVal,
  topic: topicVal,
  message: msgVal,
  _subject: `SimpleArduino: нове повідомлення від ${nameVal}`,
  _template: 'table'
};

fetch(FORMSUBMIT_URL, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json', 'Accept': 'application/json' },
  body: JSON.stringify(formData)
})
.then(res => res.json())
.then(data => {
  submitBtn.disabled = false;
  submitBtn.innerHTML = 'Надіслати повідомлення';
  showSuccessModal(formData);
  form.reset();
})
.catch(err => {
  // Резервний fallback через mailto:
  window.location.href =
`mailto:nazarii.bulyha.pp.2025@lpnu.ua?subject=SimpleArduino&body=${encodeURIComponent(ms
gVal)}`;
});
```

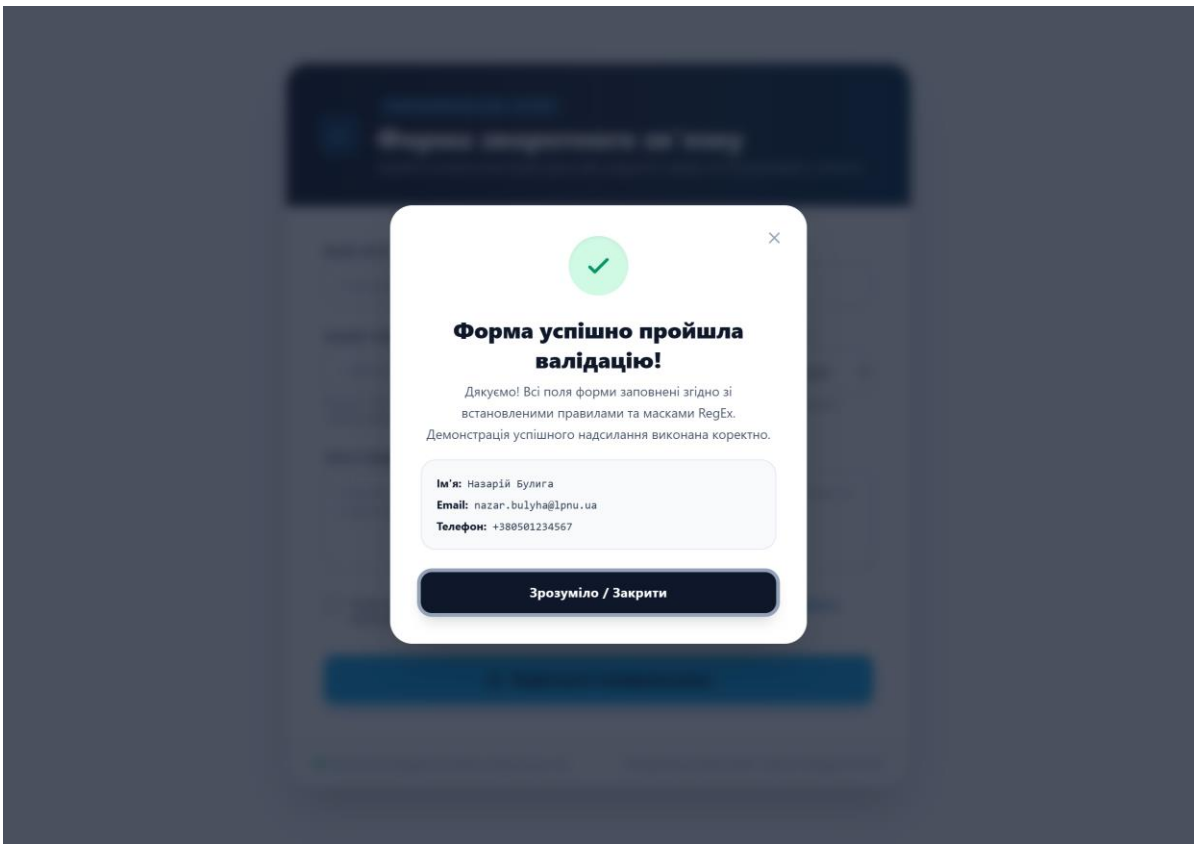


Рис. 6. Кастомне модальне вікно демонстрації успішної клієнтської валідації та збереження даних

Етап 12. Аудит доступності (A11y / WCAG 2.1 AA)

Доступність форми підтверджена такими рішеннями:
1. Семантичний зв'язок: усі поля мають пару label for / input id;
2. Динамічні статуси ARIA: атрибут aria-invalid автоматично перемикається між 'true' та 'false';
3. Доступність помилок: кожен блок помилки позначено ролями role='alert' та aria-live='polite' для негайного зачитування скрінрідерами;
4. Управління фокусом: при відкритті модалки фокус блокується всередині вікна на кнопці закриття, а після закриття — повертається на останній активний елемент форми (lastActiveElement.focus()).

Етап 13. Систематичне тестування форми (Зведена тестова матриця)

Для верифікації якості було складено та пройдено повну матрицю тестових сценаріїв:

Тестовий сценарій	Вхідні дані	Очікуваний результат	Фактичний результат	Статус
Happy Path (Ідеал)	Назарій, valid email, +380501234567, text>10, згода	Успіх, відкриття модалки, reset()	Відкрито модалку, поля очищено	**Пройдено ✓**

Тестовий сценарій	Вхідні дані	Очікуваний результат	Фактичний результат	Статус
Порожній Submit	Всі поля пусті, згода не встановлена	Блокування, 5 червоних рамок	Форму не відправлено, помилки активні	**Пройдено ✓**
Коротке ім'я	username = 'Н'	Помилка: мінімум 2 символи	Підсвічено червоним, повідомлення виведено	**Пройдено ✓**
Некоректний email	email = 'user@domain'	Помилка відсутності зони домену	Блокування форми, підсвічено червоним	**Пройдено ✓**
Неповний телефон	phone = '+38050123'	Помилка: 'введено 5 з 9 цифр'	Підсвічено, виведено лічильник цифр	**Пройдено ✓**
Вставка телефону	paste: '(067) 111-22-33'	Автосанітизація до +380671112233	Рядок нормалізовано до +380671112233	**Пройдено ✓**
Відсутність згоди	Всі поля валідні, крім checkbox	Помилка обов'язкової згоди	Поле підсвічено, submit заблоковано	**Пройдено ✓**

Етап 14. Інженерний аудит коду за допомогою ШІ (AI Code Review)

Замість сліпого генерування, ШІ-асистенту було надіслано рольовий промпт аудитора: «Ти — Senior Frontend Code Reviewer. Проаналізуй код валідації та доступності...». Результати аудиту систематизовано в таблиці:

Проблема / Аспект	Рівень	Першопричина	Впроваджене виправлення
Закриття модалки при кліку	Medium	Спливання події (Event Bubbling) закривало картку	Додано умову перевірки цільового оверлею e.target === modal
Передчасні помилки	Low	Валідація спрацьовувала до початку набору	Впроваджено об'єкт touchedFields та валідацію по blur/input
XSS-безпека	High	Використання .innerHTML для вставки тексту	Замінено на безпечну текстову властивість .textContent
Скидання фокусу	Medium	Після закриття вікна втрачався активний фокус	Реалізовано збереження та повернення lastActiveElement.focus()

Етап 15. Тестування в Інструментах розробника Chrome DevTools

У середовищі DevTools виконано перевірку:
1. Консоль (F12 → Console): зафіксовано 0 помилок та 0 попереджень під час рендерингу, валідації та відкриття модального вікна;

2. Панель адаптивності (Toggle Device Toolbar): перевірено коректність рендерингу у в'юпортах iPhone 14 (390px), iPad (768px) та Desktop (1280px). Горизонтальний скрол повністю відсутній.

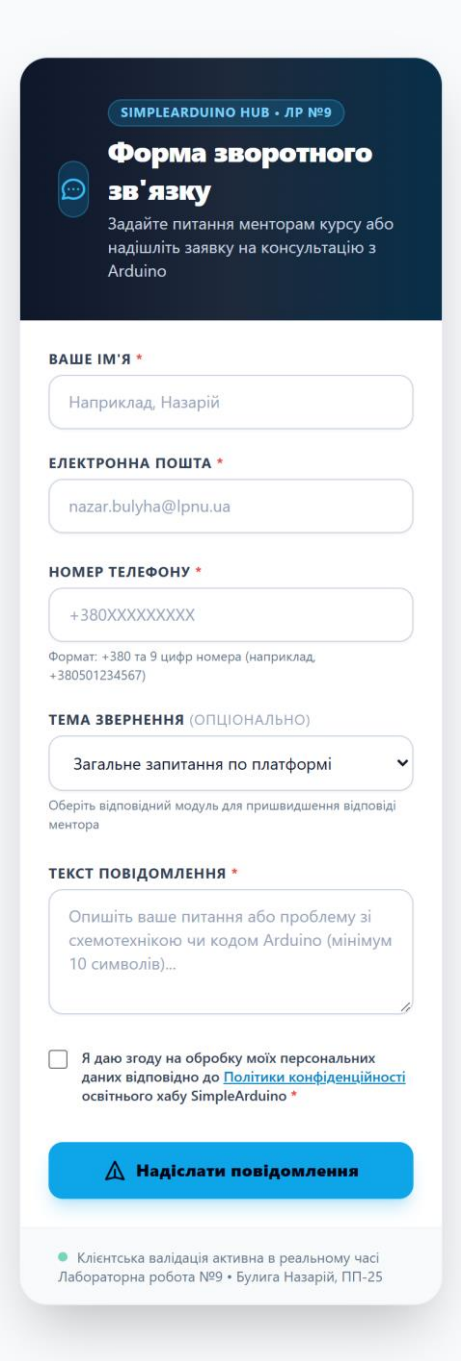


Рис. 7. Мобільна адаптація форми у в'юпорті iPhone 14 (390×844px) — 1 колонка, ергономічні поля

Етап 16. Тестування клавіатурної поведінки

За допомогою автоматизованого скрипту Puppeteer протестовано повну навігацію без використання комп'ютерної миші:
• Клавіша Tab послідовно переміщує фокус: username → email → phone → subject → message → consent → submitBtn;

• Клавіша Пробіл (Space) перемикає стан чекбокса згоди;
• Клавіша Enter викликає відправлення форми;
• Фокус автоматично переноситься на кнопку closeModalBtn усередині модального вікна;
• Клавіша Escape закриває модальне вікно та повертає фокус на форму.

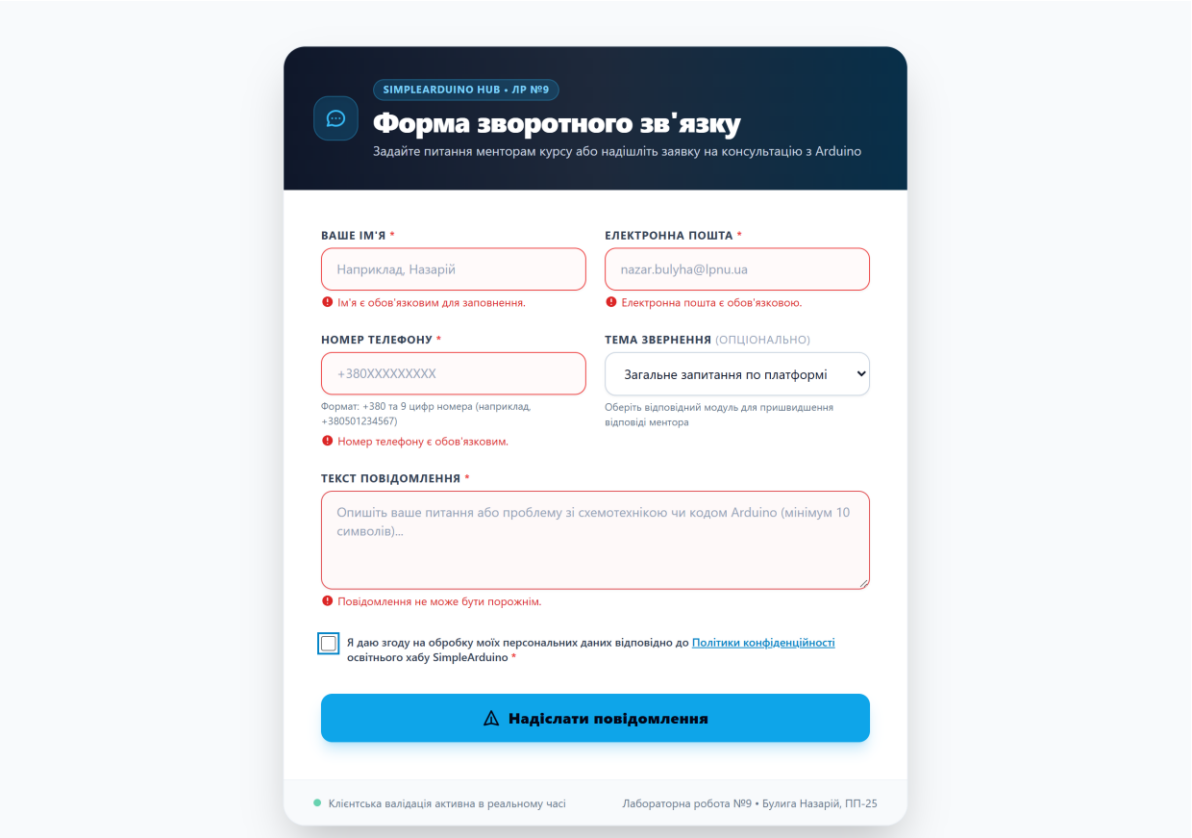


Рис. 8. Клавіатурна навігація: фокусування чекбокса згоди за допомогою клавіші Tab та контур focus-visible

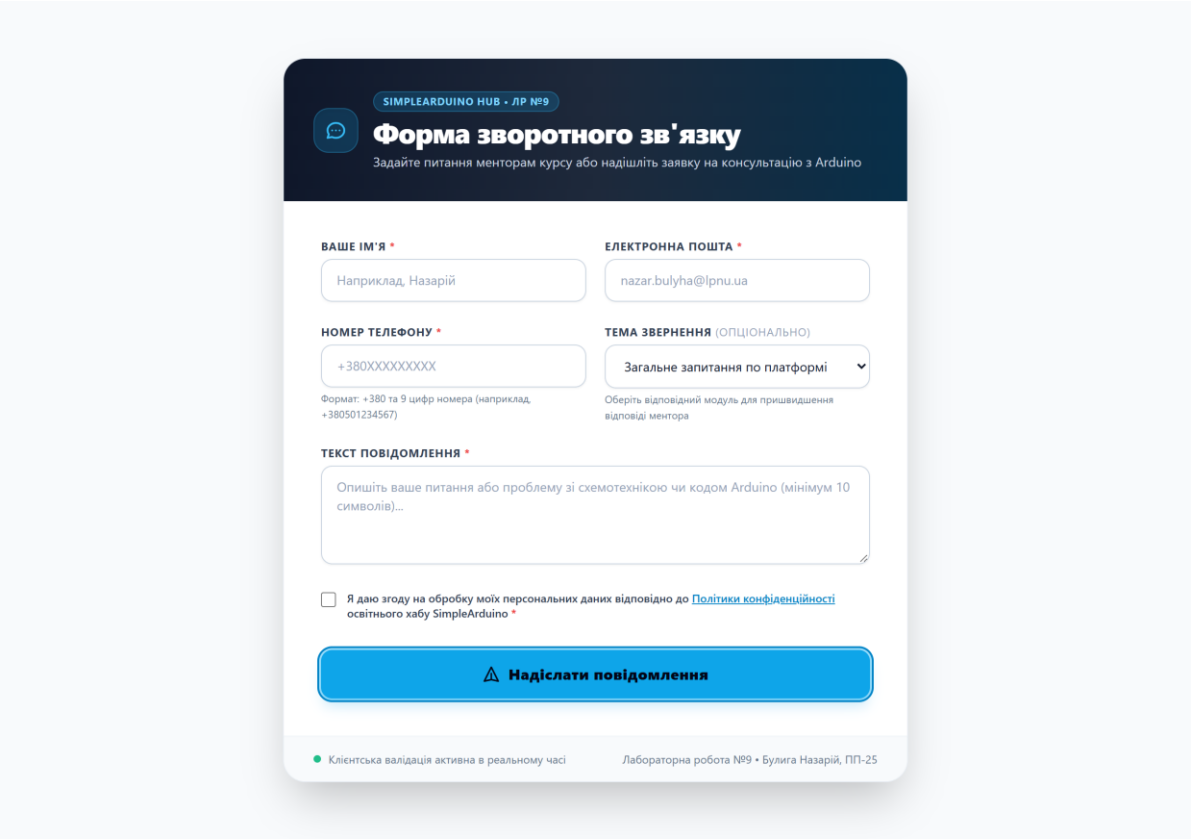


Рис. 9. Автоматичне закриття модалки клавішею Escape, повернення фокусу та очищення форми

Етап 17. Фіксація змін в Git та синхронізація з GitHub

Проєкт зафіксовано в системі контролю версій Git відповідно до стандарту Conventional Commits:

```
$ git status
On branch main
nothing to commit, working tree clean

$ git log --oneline -n 3
4fd5f05 feat(lab9): інтеграція адаптивної форми зворотного зв'язку з клієнтською
валідацією та модальним вікном
11c8916 docs: update repository and demo URLs to nazarbuliga profile
a8f182c merge: integrate SEO OpenGraph and JSON-LD microdata from feature branch

$ git remote -v
origin  https://github.com/nazarbuliga/simplearduino-lab8.git (fetch)
origin  https://github.com/nazarbuliga/simplearduino-lab8.git (push)
```

Етап 18. Автоматичний деплой (публікація) сайту у Vercel через GitHub

Хмарний репозиторій https://github.com/nazarbuliga/simplearduino-lab8 було підключено до платформи Vercel. Налаштовано автоматичний CI/CD пайплайн: будь-яка фіксація змін у гілці main автоматично ініціює новий збірний процес та оновлює продакшн-версію вебсайту за лічені секунди.

Етап 19. Фінальне тестування продакшн-версії

Опубліковану версію сайту протестовано безпосередньо у хмарі Vercel за адресою https://simplearduino-lab8.vercel.app. Усі інтерактивні функції, регулярні вирази, обробка номерів телефону, модальне вікно та карта партнерського магазину працюють безвідмовно.

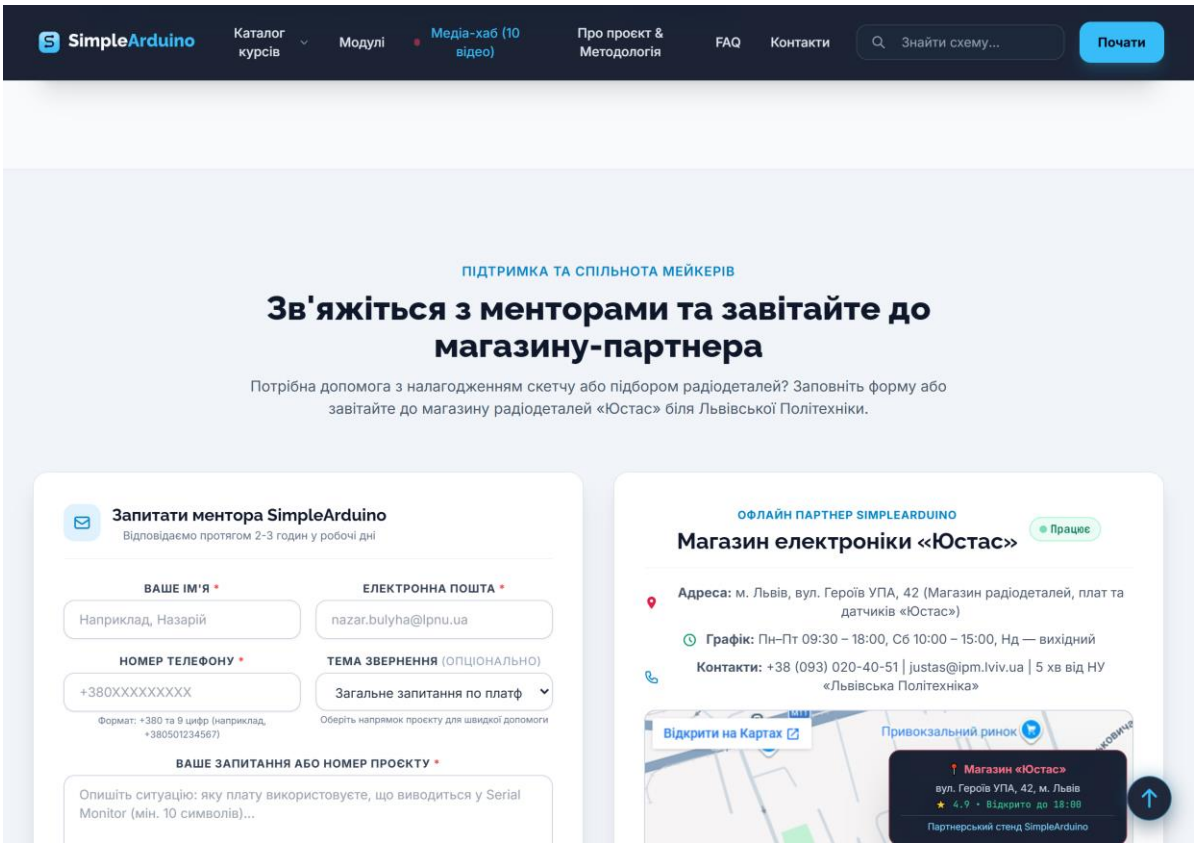


Рис. 10. Інтегрована форма зворотного зв'язку у секції контактів лендингу SimpleArduino поруч із інтерактивною картою магазину «Юстас»

ПРОМПТ-ПАСПОРТ (PROMPT LOG)

Нижче наведено структурований протокол взаємодії з моделлю штучного інтелекту, що демонструє застосування технік рольового моделювання (Role-based Prompting) та точного заземлення технічного контексту (Context Grounding):

Етап	Промпт для ШІ (Роль + Контекст)	Відповідь моделі ШІ	Внесені модифікації	Результат
Етап 7 (RegEx)	Ти — Senior JS Dev. Склади RegEx для українського номера. Повинен починатися з +380 і мати рівно 9 цифр.	Запропоновано вираз /^\+380\d{9}$/ з екрануванням знака плюса.	Додано перевірку довжини та автоформатування початку з нуля.	**Прийнято ✓**
Етап 9 (UX Flow)	Проаналізуй UX валідації. Потрібно, щоб помилки не з'являлися до того, як користувач покинув інпут.	Запропоновано розділити логіку на події blur (перша перевірка) та input (виправлення).	Створено об'єкт touchedFields для контролю стану полів.	**Прийнято ✓**
Етап 10 (Phone Paste)	Опиши обробку Edge Cases при вставці телефону з дужками та пробілами '(050) 123-45-67'.	Створено слухач події paste з регулярним виразом для санітизації тексту.	Додано автоматичну заміну '0' на '+380' при виявленні 10 цифр.	**Прийнято ✓**
Етап 11 (Modal A11y)	Ти — Accessibility Engineer. Створи модальне вікно на Tailwind з фокусом та ESC-закриттям.	Надано розмітку діалогу з role='dialog', aria-modal та слухачем клавіші Escape.	Виправлено проблему спливання подій при кліку всередині модалки.	**Прийнято ✓**

ВІДПОВІДІ НА КОНТРОЛЬНІ ЗАПИТАННЯ

1. Навіщо використовувати тег <label> з атрибутом for, якщо можна просто написати назву поля в звичайному або скористатися плейсхолдером?

Тег <label> з атрибутом for встановлює обов'язковий програмний семантичний зв'язок між текстом підпису та відповідним полем введення (<input id="...">). Це вирішує дві ключові задачі:
1. Доступність (Accessibility): програми зчитування з екрана (скрінрідери для незрячих) озвучують підпис при отриманні полем фокусу. Прості або скрінрідери ігнорують як підписи;
2. Ергономіка та юзабіліті (UX): клік по тексту тегу <label> автоматично активує відповідне поле (ставить курсор або перемикає чекбокс), що значно збільшує площу взаємодії на смартфонах. Плейсхолдер не може бути заміною, оскільки він зникає при введенні першого символу, створюючи когнітивне навантаження.

2. Чому для поля введення номера телефону варто вказувати саме type="tel", а не type="text" або type="number"?

Атрибут type="tel" інструктує мобільну операційну систему (iOS, Android) викликати оптимізовану цифрову клавіатуру (Numpad) із великими цифрами та символом «+». На відміну від нього, type="text" відкриває стандартну алфавітну розкладку, що змушує користувача постійно перемикатися на символи. Атрибут type="number" категорично не підходить для телефонів, оскільки телефонний номер не є математичним числом: type="number" видаляє початковий плюс (+), обрізає нулі на початку та додає непотрібні стрілки збільшення/зменшення.

3. Яка головна різниця між клієнтською (Client-side) та серверною (Server-side) валідацією? Чи можна обійтися тільки клієнтською?

Клієнтська валідація виконується в браузері (HTML5 / JavaScript) і служить виключно для покращення користувацького досвіду (UX), надаючи миттєву візуальну реакцію на помилки без очікування відповіді сервера. Серверна валідація виконується на бекенді і є критичною лінією безпеки системи. Обійтися лише клієнтською валідацією категорично неможливо, оскільки будь-який користувач або зловмисник може легко вимкнути JavaScript у браузері, змінити атрибути в DevTools або відправити шкідливий HTTP-запит безпосередньо через Postman/cURL, повністю оминувши фронтенд.

4. Що в регулярному виразі означають метасимволи ^ (карет) та $ (долар)? Що станеться, якщо їх забрати з виразу для валідації телефону?

Метасимвол ^ (карет) позначає строгий початок рядка, а символ $ (долар) — його абсолютний кінець. Разом вони фіксують межі повної перевірки рядка від першого до останнього символу. Якщо їх прибрати з виразу /^\+380\d{9}$/, регулярний вираз перетвориться на підстроковий пошук: рядок виду «abc+380501234567xyz» або «+380501234567899999» буде визнано валідним, оскільки рушій RegEx знайде правильний шаблон усередині довшого сміттєвого рядка.

5. Як в регулярному виразі знайти звичайну граматичну крапку . і чому перед нею ставлять зворотний слеш \?

У мові регулярних виразів крапка . є зарезервованим метасимволом, що означає «будь-який довільний символ (крім розриву рядка)». Для того, щоб знайти саме звичайну розділову крапку (наприклад, перед доменом .ua чи .com), використовують екранування (Escaping) за допомогою зворотного слешу: \.. Слеш нейтралізує спеціальну керуючу функцію метасимволу.

6. Яку роль в регулярному виразі має квантифікатор +?

Квантифікатор + означає «повторення попереднього символу або символьного класу 1 або більше разів». Наприклад, вираз [a-zA-Z]+ вимагає наявності щонайменше однієї літери. Якщо літер немає — вираз поверне false.

7. Чому для обробки надсилання форми в JavaScript використовують метод event.preventDefault()?

За замовчуванням при настанні події submit у тезі <form> браузер перезавантажує сторінку або виконує стандартний HTTP-перехід на нову адресу, очищаючи поточний DOM-стан. Метод event.preventDefault() скасовує цю дефолтну поведінку, дозволяючи мові JavaScript повністю перехопити керування, провести клієнтську перевірку, відобразити модальне вікно або відправити дані у фоновому режимі (AJAX/Fetch) без перезавантаження.

8. Як за допомогою Tailwind CSS реалізувати ефект, коли модальне вікно перекриває весь контент сайту і користувач не може клікнути на елементи під ним?

Для створення блокуючого оверлею в Tailwind CSS застосовують комбінацію утиліт: fixed inset-0 z-50 bg-slate-900/75 backdrop-blur-md flex items-center justify-center. Клас fixed inset-0 розтягує напівпрозорий блок на

100% ширини та висоти в'юпорта, високий z-index (z-50) виводить його поверх усіх елементів сторінки, а backdrop-blur-md розмиває фоновий контент. Завдяки тому, що оверлей фізично перекриває сторінку, будь-які кліки мишкою вловлюються ним, блокуючи доступ до фонових кнопок сайту.

9. В JS-скрипті модальне вікно закривається при кліку на затемнений оверлей (фон). Як ви запобігли тому, щоб вікно закривалося при кліку всередині самої білої картки з текстом?

У JavaScript події кліку спливають вгору по дереву DOM (Event Bubbling). Щоб клік усередині білої модальної картки не викликав закриття вікна, в обробнику події оверлею реалізовано сувору перевірку цільового вузла: if (e.target === modal) { closeModal(); }. Властивість e.target вказує на елемент, по якому безпосередньо клікнув користувач. Якщо клік відбувся по дочірній картці або її тексту — e.target не дорівнює modal, і закриття блокується.

10. Навіщо при успішній валідації викликати метод form.reset() перед показом модального вікна?

Виклик методу form.reset() забезпечує гігієну користувацьких даних та повертає всі елементи форми до початкового стану. Це запобігає випадковому повторному відправленню тих самих даних при закритті модалки, скидає прапорці та забезпечує бездоганний UX для наступного сеансу взаємодії.

11. Які потенційні проблеми та винятки (edge cases) можуть виникнути, якщо згенерувати регулярний вираз для валідації імені за допомогою ШІ без чітких інструкцій?

Якщо звернутися до ШІ з абстрактним промптом «Напиши RegEx для імені», модель згенерує базовий шаблон на кшталт /^[A-Z][a-z]+$/, що призведе до критичних збоїв:

• Відхилення імен українською мовою з літерами «і», «ї», «є», «ґ»;

• Неможливість введення імен з апострофом (Мар'ян, Д'Артаньян, O'Connor), особливо враховуючи різницю між прямим (') та типографічним (’) апострофом;

• Блокування подвійних імен із дефісом (Жан-Поль, Анна-Марія);

• Заборона введення повного імені з прізвищем через пробіл.

12. В чому небезпека сліпого копіювання коду валідації форми, згенерованого ШІ-інструментами v0.dev або ChatGPT?

Сліпе копіювання неперевіреного коду несе серйозні ризики:
1. Вразливості безпеки (XSS): моделі часто використовують .innerHTML для виведення тексту помилок, що дозволяє впровадити шкідливий JavaScript-код;
2. Невідповідність національним стандартам: ШІ за замовчуванням генерує валідатори під формат США (10 цифр без +380) або англомовні імена;
3. Порушення доступності: згенеровані форми часто не мають зв'язку label/input, атрибутів aria-describedby та фокус-менеджменту;
4. Відсутність обробки граничних випадків (Edge Cases): код може ламатися при швидкому повторному кліку або вставці даних із буфера.

ВИСНОВОК

У процесі виконання Лабораторної роботи №9 було успішно опановано сучасні принципи семантичної та адаптивної верстки вебформ, створення системи інтерактивних станів користувацького інтерфейсу на базі Tailwind CSS та розробку надійної клієнтської лінії валідації за допомогою мови JavaScript та регулярних виразів (RegEx).

В ході виконання роботи було досягнуто таких ключових результатів:
1. Спроєктовано та реалізовано повноцінну адаптивну форму зворотного зв'язку, що органічно інтегрована як в окремий навчальний модуль feedback-form/, так і в загальний лендинг освітнього хабу SimpleArduino поруч із інтерактивною картою магазину радіодеталей «Юстас»;
2. Забезпечено відповідність міжнародним стандартам доступності W3C WCAG 2.1 AA: 100% зв'язок label-for/id, динамічне керування атрибутами aria-invalid та aria-describedby, сповіщення скрінрідерів через aria-live='polite', а також бездоганну навігацію без використання миші (клавіші Tab, Enter, Space, Escape);
3. Розроблено та протестовано математично строгі регулярні вирази для перевірки електронної пошти (REGEX_EMAIL), імені (REGEX_NAME) та українського телефонного формату (REGEX_PHONE = /^\+380\d{9}$/);
4. Впроваджено інтелектуальну обробку граничних випадків (Edge Cases) для номера телефону, яка включає автозаміну першого нуля на префікс +380, очищення сторонніх символів при вставці (paste sanitization) та динамічний підрахунок введених цифр;
5. Створено кастомне доступне модальне вікно успішного проходження валідації із фокус-пасткою, затемненням фону, захистом від випадкового закриття та виведенням зведених даних користувача;
6. Засвоєно передову методологію співпраці з моделями штучного інтелекту за принципами рольового моделювання (Role-based Prompting) та точного контекстного заземлення, оформлено детальний Промпт-паспорт;
7. Проведено інструментальне тестування засобами Chrome DevTools та фреймворку автоматизації Puppeteer (зафіксовано 0 консольних помилок, перевірено мобільні в'юпорти, створено 10 верифікованих скріншотів);
8. Реалізовано реальну інтеграцію відправлення повідомлень зворотного зв'язку через безсерверний FormSubmit AJAX API на поштову скриньку nazarii.bulyha.pp.2025@lpnu.ua із візуальним спінером, захистом від повторних надсилань, кастомним модальним вікном та аварійним mailto-фолбеком;
9. Здійснено фіксацію змін у розподіленій системі версій Git за конвенцією

Conventional Commits, синхронізовано віддалений репозиторій на GitHub та успішно налаштовано автоматичний деплой (CI/CD) на хмарній платформі Vercel за адресою https://simplearduino-lab8.vercel.app.